Smart Customer Support Agent with Governance, Human-in-the-Loop & Evaluation Framework

1. Background & Context

DataKernels builds high-reliability AI solutions for enterprise clients. In this assessment, you are tasked with designing and building a prototype for a **Smart Customer Support Agent** capable of processing inbound customer requests, retrieving policy information, classifying intents, applying safety and escalation guardrails, and evaluating its own accuracy.

This exercise is designed to evaluate your system architecture skills, business acumen, prompt engineering, output structuring, and production-mindedness (testing, metrics, and risk management).

2. Functional Requirements

Your submission should implement a pipeline that performs the following steps:

1. **Ingest & Parse:** Process incoming customer tickets containing unstructured text.
2. **Context Retrieval (RAG):** Query a local knowledge base of company policies to extract context relevant to the user query.
3. **Structured Classification & Drafting:** Utilize an LLM (via JSON mode, Pydantic, or function calling) to output structured data including:
    - Intent category
    - Detected sentiment
    - Extracted entities (Order ID, dollar amounts, dates)
    - Confidence score (0.0 to 1.0)
    - Drafted customer response
4. **Human-in-the-Loop (HITL) & Escalation Routing:** Automatically evaluate whether the ticket can be resolved automatically or must be escalated based on deterministic business rules
    - **Low Confidence:** Model confidence score < 0.75
    - **Financial Threshold:** Disputed amount $> \$100$
    - **Knowledge Gap:** Retrieval similarity score < 0.60
    - **High Risk / Churn:** Legal threat or extreme customer frustration
5. **Evaluation Framework:** Define and implement a basic automated testing script that checks schema validity, hallucination rate, or intent classification metrics against a sample test set.

3. Mock Data Schemas

You are required to create mock datasets adhering to the following structures:

A. Customer Tickets (`support_tickets`)

- `ticket_id` *(String)*
- `customer_id` *(String)*
- `raw_message` *(Text)*
- `timestamp` *(DateTime)*
-

B. Policy Knowledge Base (`policy_kb`)

- `policy_id` *(String)*
- `category` *(String)* (e.g., Returns, Billing, Technical, Shipping)
- `policy_text` *(Text)*

C. Required Enriched Output Schema

JSON

None

```
{
  "ticket_id": "TCK-8821",
  "category": "Billing Dispute",
  "sentiment": "Frustrated",
  "extracted_entities": {
    "order_id": "ORD-9910",
    "disputed_amount": 149.99
  },
  "confidence_score": 0.82,
  "human_in_loop_required": true,
  "escalation_reason": "Refund amount exceeds $100 automated policy cap",
  "auto_response_draft": "Hello, I have reviewed your request regarding Order #ORD-9910. Because the refund amount ($149.99) requires manager approval under our billing policy, I have forwarded this directly to our senior support team for priority processing."
}
```

4. Evaluation & Metrics Criteria

In your documentation or presentation, address how you evaluate and monitor the system mathematically ??

5. Candidate Deliverables

Please submit the following items upon completion:

1. Source Code Repository

- A public GitHub/GitLab link (or zip archive) containing clean, modular Python code.
- Environment configuration instructions (`.env.example`, `requirements.txt` or `Poetry`/`Pipenv`).
- Structured Pydantic/JSON output validation logic.
- An automated test suite/script (`pytest` or standalone script) validating schema integrity and escalation triggers.

2. Running Demo & Recorded Walkthrough (5–8 minutes)

- A link to a short screen recording (Loom, YouTube unlisted, or Google Drive) demonstrating:
    - A live walk-through of the code architecture and data flow.
    - Executing the script/pipeline with sample test cases (showing both automated resolution and human-escalation triggers).
    - Explanation of your RAG strategy, guardrail triggers, and testing setup.

3. System Documentation (`README.md`)

- Setup and execution instructions.
- Architecture summary & technical choice justification (model choice, vector store, frame selection).
- Estimated operational cost breakdown (e.g., cost/latency per 1,000 processed tickets).

6. Next Steps & Technical Discussion Call

Once we receive and review your repository and video demonstration, we will schedule a **45-minute follow-up technical call** & Some coding questions

What to expect in the call:

- **Code Deep-Dive:** Discussing specific implementation choices, error handling, and prompt structures.
- **System Design & Scaling:** How you would scale this pipeline from 100 tickets/day to 100,000 tickets/day in a production cloud environment (AWS/GCP).
- **Live Scenario Adjustments:** Exploring how you would adapt the system under changing business rules or budget constraints.